Java for Automation Testing Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**335**
Exact duplicates removed (same question, different punctuation, case or formatting)	**61**
Near-duplicates merged (same question asked in different words)	**158**
Unique questions in this PDF	**116**

Section 1: Critical (Must Prepare)	30 questions
Section 2: High Priority	48 questions
Section 3: Medium Priority	26 questions
Section 4: Low Priority / Quick Revision	12 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 30 questions	Asked in almost every interview at 4+ years. A weak answer here usually decides the outcome, and interviewers follow up with examples from your own project. Prepare a crisp definition plus a real-life example for each.
High Priority 48 questions	Asked frequently, mostly as process, scenario and 'how would you handle' questions. These separate an experienced tester from a fresher, so answer with a structured approach (steps, risks, trade-offs).
Medium Priority 26 questions	Breadth topics. Asked occasionally, usually as follow-ups to your earlier answers or to probe a specific area on your resume. One clear paragraph per question is enough.
Low Priority / Quick Revision 12 questions	Fresher-level definitions and niche topics. Rarely asked at 4+ years, but worth a one-line answer so you are never caught off guard.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
OOP Concepts	Can you explain each pillar with a real example from YOUR framework (page objects, base classes, drivers)?
Collections Framework	Internal working (HashMap), choosing the right collection, and where you used it in automation.
Strings & equals/hashCode	Depth on the classics: immutability, String pool, == vs equals, the hashCode contract.
Exception Handling	Do you handle failures properly in a framework: custom exceptions, finally, checked vs unchecked?
Java in Automation Frameworks	Design thinking: POJOs, Jackson/Gson, BaseTest, config files, singleton, design patterns.
Core Language	Precision on static, final, access modifiers and pass-by-value.
Tricky "Can We...?"	Quick-fire trap questions: they test whether your fundamentals are exact, not memorized.
Java 8	Comfort with lambdas and streams for cleaner test code and data handling.
Multithreading & GC	Parallel execution in frameworks and basic knowledge of memory and thread safety.
Arrays & Coding	Small live-coding tasks and array/list edge cases.

Note: a few one-line entries in your original file (for example 'Instance variables' or 'Design pattern in JAVA') were topics, not full questions. They have been phrased as complete questions. Your file had no live-coding problems beyond linked-list reversal and array checks.

Table of Contents

Section 1: Critical (Must Prepare) (30 questions) 5

OOP Concepts (9) 5
Collections Framework (8) 6
Strings, Immutability & equals/hashCode (4) 7
Exception Handling (4) 7
Java in Automation Frameworks (3) 8
Core Language: static, final, Access Modifiers & Basics (2) 8

Section 2: High Priority (48 questions). 10

OOP Concepts (8) 10
Collections Framework (10) 11
Strings, Immutability & equals/hashCode (1) 12
Exception Handling (5) 12
Java in Automation Frameworks (9) 13
Core Language: static, final, Access Modifiers & Basics (3) 15
Tricky "Can We...?" Questions (6) 15
Java 8: Lambda, Functional Interfaces & Streams (3) 16
Multithreading, Synchronization & Garbage Collection (3) 16

Section 3: Medium Priority (26 questions). 18

OOP Concepts (4) 18
Collections Framework (3) 18
Java in Automation Frameworks (4) 19
Core Language: static, final, Access Modifiers & Basics (4) 19
Tricky "Can We...?" Questions (4) 20
Java 8: Lambda, Functional Interfaces & Streams (2) 20
Multithreading, Synchronization & Garbage Collection (2) 20
Arrays & Coding Problems (3) 21

Section 4: Low Priority / Quick Revision (12 questions) 22

Collections Framework (1) 22
Core Language: static, final, Access Modifiers & Basics (4) 22
Tricky "Can We...?" Questions (1) 22
Java 8: Lambda, Functional Interfaces & Streams (1) 23
Arrays & Coding Problems (5) 23

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
OOP Concepts	9 Qs, p.5	8 Qs, p.10	4 Qs, p.18	-	**21**
Collections Framework	8 Qs, p.6	10 Qs, p.11	3 Qs, p.18	1 Qs, p.22	**22**
Strings, Immutability & equals/hashCode	4 Qs, p.7	1 Qs, p.12	-	-	**5**
Exception Handling	4 Qs, p.7	5 Qs, p.12	-	-	**9**
Java in Automation Frameworks	3 Qs, p.8	9 Qs, p.13	4 Qs, p.19	-	**16**
Core Language: static, final, Access Modifiers & Basics	2 Qs, p.8	3 Qs, p.15	4 Qs, p.19	4 Qs, p.22	**13**
Tricky "Can We...?" Questions	-	6 Qs, p.15	4 Qs, p.20	1 Qs, p.22	**11**
Java 8: Lambda, Functional Interfaces & Streams	-	3 Qs, p.16	2 Qs, p.20	1 Qs, p.23	**6**
Multithreading, Synchronization & Garbage Collection	-	3 Qs, p.16	2 Qs, p.20	-	**5**
Arrays & Coding Problems	-	-	3 Qs, p.21	5 Qs, p.23	**8**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (30 questions)

Asked in almost every interview at 4+ years. A weak answer here usually decides the outcome, and interviewers follow up with examples from your own project. Prepare a crisp definition plus a real-life example for each.

OOP Concepts (9)

1. What is the difference between method overloading and method overriding? Give examples (e.g. from your Selenium framework). **[asked x12 in your bank]**

   > **Answer:** Overloading: same method name with different parameters in the same class, resolved at compile time. Overriding: a subclass redefines a parent method with the same signature, resolved at runtime.
   >
   > **Example:** Overloading: addToCart(sku) and addToCart(sku, qty) in a CartPage. Overriding: PaymentPage.pay() redefined in UpiPaymentPage.

   ```
   void addToCart(String sku) { }
   void addToCart(String sku, int qty) { }   // overloading
   @Override void pay() { }                  // overriding in child class
   ```

2. What is the difference between an Abstract Class and an Interface? When do you use each? **[asked x11 in your bank]**

   > **Answer:** An abstract class can hold state, constructors and both abstract and concrete methods; a class extends only one. An interface is a contract (default/static methods allowed); a class can implement many. Use an abstract class for shared code, an interface for a shared capability.
   >
   > **Example:** abstract BasePage holds common waits and driver; interface PaymentMethod is implemented by CardPayment, UpiPayment and WalletPayment.

3. Explain the four OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction) with real examples and where you applied them in your framework. **[asked x12 in your bank]**

   > **Answer:** Encapsulation: hide data behind methods. Inheritance: reuse parent behaviour. Polymorphism: one reference, many forms. Abstraction: expose what, hide how.
   >
   > **Example:** Page Object: private locators with public methods (encapsulation); LoginPage extends BasePage (inheritance); WebDriver d = new ChromeDriver() (polymorphism); interfaces such as PaymentMethod (abstraction).

4. What is Polymorphism? Where do you use it? **[asked x6 in your bank]**

   > **Answer:** Polymorphism means one interface with many implementations. Compile-time (overloading) and runtime (overriding) are the two types. It makes code flexible and loosely coupled.
   >
   > **Example:** WebDriver driver = new ChromeDriver() or new FirefoxDriver(): the same driver.get() call runs on any browser.

5. What is the difference between compile-time and runtime polymorphism (static vs dynamic binding)? **[asked x3 in your bank]**

   > **Answer:** Compile-time (static binding): overloading, the method is chosen by the compiler from the argument types. Runtime (dynamic binding): overriding, the JVM picks the method by the actual object type.
   >
   > **Example:** search(String) vs search(String, double maxPrice) is compile-time. Product p = new Electronics(); p.getTax() calling Electronics' version is runtime.

6. What is Encapsulation and how is it achieved? Explain with an example. **[asked x3 in your bank]**

   > **Answer:** Encapsulation binds data and methods together and restricts direct access. Achieve it with private fields plus public getters/setters.
   >
   > **Example:** A Product class keeps price private and exposes getPrice(); setPrice() rejects negative values.

   ```
   class Product {
     private double price;
     public double getPrice() { return price; }
     public void setPrice(double p) { if (p >= 0) price = p; }
   }
   ```

7. What is the difference between Abstraction and Encapsulation? **[asked x3 in your bank]**

> **Answer:** Abstraction hides complexity at the design level (what the object does). Encapsulation hides data at the implementation level (protects state). Abstraction is achieved with abstract classes/interfaces, encapsulation with access modifiers.
>
> **Example:** Abstraction: PaymentMethod.pay() hides how payment works. Encapsulation: card number stays private in CardPayment.

8. What is Abstraction and how do you implement it in Java? **[asked x2 in your bank]**

> **Answer:** Abstraction shows only essential behaviour and hides implementation. In Java it is implemented with abstract classes (partial) and interfaces (full).
>
> **Example:** interface PaymentMethod { void pay(double amount); } with CardPayment and UpiPayment providing the logic.

9. What is Inheritance? Explain with an example. **[asked x5 in your bank]**

> **Answer:** Inheritance lets a child class acquire the fields and methods of a parent using extends, giving code reuse and an is-a relationship.
>
> **Example:** LoginPage, CartPage and CheckoutPage extend BasePage, so all of them reuse waitForElement() and click().
>
> ```
> class BasePage { void click(By by) { } }
> class CartPage extends BasePage { }   // reuses click()
> ```

Collections Framework (8)

10. How does HashMap work internally? How is it implemented using key-value pairs? **[asked x3 in your bank]**

> **Answer:** HashMap uses an array of buckets. hashCode() of the key gives the bucket index; collisions are chained as a linked list, converted to a tree when a bucket grows large (8+). equals() finds the exact key. It resizes at 0.75 load factor.
>
> **Example:** Map<String,Integer> cart: put("SKU101", 2) hashes the key to a bucket; get("SKU101") finds it by hash and equals().

11. What is the difference between HashMap and HashSet? **[asked x3 in your bank]**

> **Answer:** HashMap stores key-value pairs. HashSet stores only unique elements and is internally backed by a HashMap (elements act as keys).
>
> **Example:** HashMap: sku -> quantity in the cart. HashSet: unique product categories on the search page.

12. What is the difference between ArrayList and LinkedList? Which is better for modification? (also: List<String> list = new ArrayList<>() vs ArrayList<String> list = new ArrayList<>()) **[asked x7 in your bank]**

> **Answer:** ArrayList is a resizable array: fast random access, slow insert/delete in the middle. LinkedList is a doubly linked list: fast insert/delete at ends or via iterator, slow get(i). List<String> l = new ArrayList<>() codes to the interface so the implementation can be swapped.
>
> **Example:** ArrayList for reading a product grid by index; LinkedList for a queue of orders where items are added and removed often.

13. What is the difference between List, Set and Map? **[asked x6 in your bank]**

> **Answer:** List: ordered, allows duplicates. Set: no duplicates. Map: key-value pairs with unique keys.
>
> **Example:** List: product names as displayed. Set: unique brands in the filter panel. Map: sku -> price.

14. What is the difference between HashMap, LinkedHashMap and TreeMap? **[asked x4 in your bank]**

> **Answer:** HashMap: no order, fastest. LinkedHashMap: keeps insertion order. TreeMap: sorted by key (natural or Comparator), O(log n), no null keys.
>
> **Example:** TreeMap<String,Double> of price by product name to verify alphabetical sorting; LinkedHashMap to keep checkout steps in order.

15. What is the difference between Comparable and Comparator? **[asked x4 in your bank]**

> **Answer:** Comparable defines the natural order inside the class via compareTo(). Comparator defines external, multiple orders via compare().
>
> **Example:** Sort products by price (natural order in Product) or by rating/name using separate Comparators to validate the sort dropdown.

```
list.sort(Comparator.comparingDouble(Product::getPrice));
```

16. What is the Java Collections Framework? Explain the hierarchy. **[asked x4 in your bank]**

> **Answer:** A set of interfaces and classes to store and process groups of objects. Iterable > Collection > List, Set, Queue. Map is a separate hierarchy (not a Collection).
>
> **Example:** List<WebElement> for product cards, Set for unique filters, Map for test data.

17. What is the difference between an Array and an ArrayList? **[asked x4 in your bank]**

> **Answer:** Array: fixed size, holds primitives or objects, accessed with length. ArrayList: dynamic size, objects only (wrappers), with built-in methods like add/remove/contains.
>
> **Example:** Array for a fixed set of 3 browser names; ArrayList for search results whose count varies.

Strings, Immutability & equals/hashCode (4)

18. What is the difference between String, StringBuilder and StringBuffer? **[asked x9 in your bank]**

> **Answer:** String is immutable. StringBuilder is mutable and fast but not thread-safe. StringBuffer is mutable and synchronized (thread-safe, slower).
>
> **Example:** Build a dynamic order-API URL in a loop with StringBuilder instead of creating many String objects.

```
StringBuilder sb = new StringBuilder("/orders?");
sb.append("status=").append("PAID");
```

19. Why are Strings immutable in Java? **[asked x3 in your bank]**

> **Answer:** Strings are immutable for String pool sharing, security (URLs, credentials cannot change), thread safety and hashCode caching, which makes them reliable HashMap keys.
>
> **Example:** Changing a coupon code string creates a new String; the original value other code relies on stays safe.

20. What is the difference between == and .equals()? **[asked x6 in your bank]**

> **Answer:** == compares references (or values for primitives). equals() compares content when overridden, as in String.
>
> **Example:** new String("SAVE10") == "SAVE10" is false, but .equals() is true. Use equals() to verify the applied coupon text.

```
String a = new String("SAVE10");
System.out.println(a == "SAVE10");       // false
System.out.println(a.equals("SAVE10"));   // true
```

21. Why is it important to override hashCode() when you override equals()?

> **Answer:** Contract: equal objects must return equal hash codes. If only equals() is overridden, HashMap/HashSet may place equal objects in different buckets and fail to find them or allow duplicates.
>
> **Example:** Two Product objects with the same sku are equal; without hashCode() both appear in a HashSet and the cart shows duplicates.

Exception Handling (4)

22. What is the difference between final, finally and finalize()? **[asked x7 in your bank]**

> **Answer:** final: keyword for constants, non-overridable methods and non-extendable classes. finally: block that always runs after try/catch for cleanup. finalize(): method called by GC before collecting an object (deprecated).
>
> **Example:** final String BASE_URL; finally { driver.quit(); } to close the browser even when the checkout test fails.

23. What is the difference between checked and unchecked exceptions? **[asked x6 in your bank]**

> **Answer:** Checked exceptions are verified at compile time and must be handled or declared (IOException, SQLException). Unchecked exceptions extend RuntimeException (NullPointerException, ArrayIndexOutOfBoundsException) and need no mandatory handling.
>
> **Example:** Reading product data from a file may throw checked FileNotFoundException; getting item 5 of a 3-item cart throws unchecked IndexOutOfBoundsException.

24. What is the difference between throw and throws? **[asked x4 in your bank]**

> **Answer:** throw is a statement that actually throws one exception object. throws is part of the method signature that declares exceptions the method may pass to its caller.
>
> **Example:** A method declares throws IOException when reading test data; inside it, throw new IllegalArgumentException("Invalid quantity") for bad input.

```
void loadData() throws IOException {
  if (qty < 1) throw new IllegalArgumentException("Invalid quantity");
}
```

25. What is exception handling? Explain try-catch-finally and the types of exceptions. **[asked x4 in your bank]**

> **Answer:** Exception handling manages runtime errors so the program does not stop abruptly. try holds risky code, catch handles it, finally runs cleanup. Types: checked, unchecked and Error.
>
> **Example:** Wrap the Pay Now click in try-catch for TimeoutException, take a screenshot in catch, and quit the driver in finally.

```
try { pay.click(); }
catch (TimeoutException e) { takeScreenshot(); }
finally { driver.quit(); }
```

Java in Automation Frameworks (3)

26. Explain collections and where you used them in your automation framework.

> **Answer:** Collections hold groups of data. I used List for web elements and test data, Set for unique values, Map for key-value data like config and payloads.
>
> **Example:** List<WebElement> of product cards to verify the sort order; Set to check no duplicate products; Map<String,String> for checkout form data.

27. Where did you use HashMap and OOP concepts in your automation framework?

> **Answer:** HashMap stores test data, config and request payloads by key. OOP: Page Objects (encapsulation), BasePage/BaseTest (inheritance), WebDriver reference (polymorphism), interfaces (abstraction).
>
> **Example:** A Map<String,String> holds address fields for checkout; CheckoutPage extends BasePage with private locators and public actions.

28. Why is Java used in automation testing and how is it integrated with Selenium / automation frameworks? **[asked x3 in your bank]**

> **Answer:** Java is object-oriented, platform independent, has a strong ecosystem (Selenium, TestNG, Maven, Rest Assured) and is Selenium's most supported language. Integration: add selenium-java via Maven, create a WebDriver and run tests with TestNG/JUnit.
>
> **Example:** A Maven project with selenium-java and TestNG runs the login-to-checkout suite on Chrome and Firefox.

Core Language: static, final, Access Modifiers & Basics (2)

29. What is the difference between JDK, JRE and JVM? **[asked x3 in your bank]**

> **Answer:** JVM runs bytecode and is platform specific. JRE = JVM + libraries, needed to run programs. JDK = JRE + compiler (javac) and tools, needed to develop.
>
> **Example:** A tester needs the JDK to compile the Selenium framework; the CI machine running a packaged jar needs only the JRE.

30. What are static variables and static methods? How do they differ from non-static members, and when should a method be static? **[asked x9 in your bank]**

> **Answer:** Static members belong to the class, are shared by all objects and are accessed with the class name; non-static members belong to each object. Make a method static when it needs no instance state, such as utilities.
>
> **Example:** static String randomEmail() in a DataUtil class is called as DataUtil.randomEmail() without creating an object.

```
class DataUtil {
  static String randomEmail() { return "u" + System.nanoTime() + "@x.com"; }
}
```

Section 2: High Priority (48 questions)

Asked frequently, mostly as process, scenario and 'how would you handle' questions. These separate an experienced tester from a fresher, so answer with a structured approach (steps, risks, trade-offs).

OOP Concepts (8)

31. What is a constructor? What are default, parameterized and copy constructors, and how does it differ from a method? Does Java provide a default constructor? **[asked x7 in your bank]**

> **Answer:** A constructor has the class name, no return type and runs on object creation to initialize it. Default: no-arg, supplied by the compiler only if none is written. Parameterized: takes values. Copy: creates an object from another (user-defined). Unlike methods, constructors are not inherited or called explicitly.
>
> **Example:** LoginPage(WebDriver driver) is a parameterized constructor that stores the driver for every page action.

```
class LoginPage {
  WebDriver driver;
  LoginPage(WebDriver driver) { this.driver = driver; }
}
```

32. What is constructor overloading? Can a constructor be overloaded and how does it differ from method overloading? **[asked x2 in your bank]**

> **Answer:** Constructor overloading means several constructors with different parameter lists. It differs from method overloading: constructors have no return type, are not inherited, and can call each other using this().
>
> **Example:** Product(sku) and Product(sku, price, qty) allow creating a product with minimal or full data.

33. What are the this and super keywords and what are they used for? **[asked x9 in your bank]**

> **Answer:** this refers to the current object (resolve name clashes, call another constructor with this()). super refers to the parent (call parent methods/fields, or its constructor with super()).
>
> **Example:** In CartPage(WebDriver driver), super(driver) calls BasePage's constructor and this.driver = driver assigns the field.

34. What is an abstract class, an abstract method and the abstract modifier? When would you use an abstract class? **[asked x3 in your bank]**

> **Answer:** An abstract class cannot be instantiated. An abstract method has no body and must be implemented by subclasses. Use it when classes share common code plus mandatory behaviour.
>
> **Example:** abstract class BasePage has concrete waitFor() and an abstract isPageLoaded() that each page must implement.

```
abstract class BasePage { abstract boolean isPageLoaded(); }
```

35. How and when do you use an interface? Can a class implement multiple interfaces? **[asked x3 in your bank]**

> **Answer:** An interface defines a contract that classes implement; a class can implement multiple interfaces. Use it for capabilities shared across unrelated classes.
>
> **Example:** CardPayment implements PaymentMethod, Refundable: it can both pay and refund.

36. When do you choose abstraction vs an interface? Explain loose coupling and tight coupling.

> **Answer:** Use an abstract class when classes share state/code; an interface when unrelated classes share a capability or you need multiple inheritance. Loose coupling: depend on interfaces so implementations are swappable. Tight coupling: depend on concrete classes.
>
> **Example:** Code against the WebDriver interface (loose) rather than ChromeDriver (tight) so tests run on any browser.

37. Why does Java not support multiple inheritance with classes, how is it achieved with interfaces, and what would happen if it were allowed? **[asked x4 in your bank]**

> **Answer:** Java avoids multiple class inheritance because of the diamond problem: ambiguity over which parent's method runs. Interfaces allow it since they carry no state; conflicting default methods must be overridden explicitly.
>
> **Example:** If class C extended A and B, both defining applyDiscount(), the compiler could not tell which one to run.

38. Class A has 3 methods and class B (2 methods) extends A. How do you call A's methods through an object of B?

> **Answer:** Because B extends A, create B b = new B() and call A's methods directly (b.m1()), or use super.m1() inside B. Public and protected methods are inherited.
>
> **Example:** CartPage extends BasePage; cartPage.click(locator) calls the method defined in BasePage.

```
B b = new B();
b.methodOfA();   // inherited from A
```

Collections Framework (10)

39. What is the difference between HashMap and Hashtable? **[asked x3 in your bank]**

> **Answer:** HashMap is not synchronized, allows one null key and null values, and is faster. Hashtable is synchronized (legacy), allows no nulls and is slower.
>
> **Example:** HashMap for single-threaded test data; for parallel tests prefer ConcurrentHashMap over Hashtable.

40. Is HashMap thread safe?

> **Answer:** No. HashMap is not thread safe. Use ConcurrentHashMap or Collections.synchronizedMap() when threads share it.
>
> **Example:** Parallel tests updating one shared order-ID map can corrupt it; ConcurrentHashMap avoids that.

41. What is the difference between Collection and Collections? **[asked x3 in your bank]**

> **Answer:** Collection is the root interface for List, Set and Queue. Collections is a utility class with static methods such as sort(), reverse(), max() and unmodifiableList().
>
> **Example:** Collections.sort(prices) to verify a low-to-high sort on the product list.

42. What is a HashMap? Can you store objects in it and how do you retrieve them? **[asked x2 in your bank]**

> **Answer:** HashMap stores objects as key-value pairs. Add with put(key, value) and retrieve with get(key); keys must be unique.
>
> **Example:** Map<String,Integer> cart = new HashMap<>(); cart.put("SKU101", 2); cart.get("SKU101") returns 2.

43. Explain HashSet, TreeSet and LinkedHashSet and their differences. **[asked x2 in your bank]**

> **Answer:** HashSet: unique, no order, fastest. LinkedHashSet: unique, keeps insertion order. TreeSet: unique and sorted, no nulls.
>
> **Example:** TreeSet of product names to check the alphabetical filter; HashSet to detect duplicate listings.

44. Explain ArrayList, LinkedList and Vector. When do you use each? **[asked x2 in your bank]**

> **Answer:** ArrayList: dynamic array, fast reads, not synchronized. LinkedList: fast insert/delete. Vector: synchronized legacy list. Default to ArrayList.
>
> **Example:** ArrayList for search results; LinkedList for a recently viewed list where items are removed from both ends.

45. What are Generics and how do they ensure type safety? **[asked x3 in your bank]**

> **Answer:** Generics parameterize classes and methods with types, giving compile-time type safety and removing casts.
>
> **Example:** List<Product> accepts only Product objects; adding a String is a compile error instead of a runtime failure.

```
List<String> names = new ArrayList<>();
names.add("Laptop");   // names.add(10) will not compile
```

46. How do you iterate over a Collection or Map (Iterator, for-each, Map.Entry)? **[asked x4 in your bank]**

> **Answer:** Use a for-each loop, an Iterator, or for Maps entrySet() with Map.Entry (also keySet(), values() and forEach with lambda).
>
> **Example:** Loop over Map<String,Integer> cart entries to print each sku and quantity.

```
for (Map.Entry<String,Integer> e : cart.entrySet())
  System.out.println(e.getKey() + "=" + e.getValue());
```

47. How do you remove elements while iterating (Iterator)? **[asked x2 in your bank]**

> **Answer:** Use iterator.remove(). Calling list.remove() inside a for-each throws ConcurrentModificationException. removeIf() is a shorter option.
>
> **Example:** Remove out-of-stock items from the cart list while looping over it.

```
Iterator<Item> it = cart.iterator();
while (it.hasNext()) if (it.next().isOutOfStock()) it.remove();
```

48. What is the difference between fail-fast and fail-safe iterators/collections? **[asked x4 in your bank]**

> **Answer:** Fail-fast iterators throw ConcurrentModificationException if the collection changes during iteration (ArrayList, HashMap). Fail-safe iterators work on a copy and do not throw (CopyOnWriteArrayList, ConcurrentHashMap).
>
> **Example:** Removing an item from the cart list inside a for-each fails fast; the same loop on CopyOnWriteArrayList does not.

Strings, Immutability & equals/hashCode (1)

49. Why is immutability important and how do you create an immutable class? **[asked x3 in your bank]**

> **Answer:** Immutable objects are thread safe, safe as map keys and easy to reason about. Create one with a final class, private final fields, no setters, constructor initialization and defensive copies of mutable fields.
>
> **Example:** An immutable Coupon(code, percent) object cannot be changed by any test once created.

```
final class Coupon {
  private final String code;
  Coupon(String code) { this.code = code; }
  String getCode() { return code; }
}
```

Exception Handling (5)

50. Explain the exception hierarchy in Java and how to handle exceptions. **[asked x3 in your bank]**

> **Answer:** Throwable splits into Error (JVM problems) and Exception. Exception has checked exceptions and RuntimeException (unchecked). Handle with try-catch-finally, throws and try-with-resources.
>
> **Example:** IOException (checked) when reading test data; NullPointerException (unchecked) when a locator returns null.

51. How do you create and throw a custom exception?

> **Answer:** Extend Exception (checked) or RuntimeException (unchecked), add a constructor with a message, then throw it with throw new.
>
> **Example:** OutOfStockException thrown by the cart when a product has no stock.

```
class OutOfStockException extends RuntimeException {
  OutOfStockException(String msg) { super(msg); }
}
throw new OutOfStockException("SKU101 unavailable");
```

52. What are the best practices for exception handling?

> **Answer:** Catch specific exceptions, never swallow them silently, log with context, clean up with finally/try-with-resources, do not use exceptions for flow control, and use meaningful custom exceptions.
>
> **Example:** Catch NoSuchElementException on the Add to Cart button, log the locator and screenshot, then fail the test with a clear message.

53. What is finally and where do you use it?

> **Answer:** finally is a block that runs after try/catch whether or not an exception occurred (except System.exit or a JVM crash). Use it for cleanup.
>
> **Example:** driver.quit() or closing the DB connection after verifying an order, even if the verification fails.

54. Can we use multiple catch blocks? When do you use them?

> **Answer:** Yes. Use multiple catch blocks to handle different exceptions differently; order from specific to general. Multi-catch with | combines them.
>
> **Example:** Catch TimeoutException to retry the payment page and NoSuchElementException to log a locator issue.

```
try { pay(); }
catch (TimeoutException | NoSuchElementException e) { log(e); }
catch (Exception e) { throw e; }
```

Java in Automation Frameworks (9)

55. What are Serialization and Deserialization and how do you achieve them? **[asked x4 in your bank]**

> **Answer:** Serialization converts an object into a byte stream/JSON/XML to store or send; deserialization rebuilds the object. In Java use Serializable with ObjectOutputStream, or Jackson/Gson for JSON.
>
> **Example:** Serialize an Order POJO to JSON for the POST /orders request and deserialize the response back into an Order object.

56. What is a POJO, why do we use it, and can you write one for serialization/deserialization? **[asked x2 in your bank]**

> **Answer:** A POJO is a plain class with private fields, getters/setters and a constructor, with no framework dependency. It maps request/response payloads for serialization.
>
> **Example:** A Product POJO with sku, name and price maps the product API response.

```
public class Product {
  private String sku; private double price;
  public String getSku() { return sku; }
  public void setSku(String s) { sku = s; }
}
```

57. How do you use Jackson/Gson for JSON parsing?

> **Answer:** Jackson: ObjectMapper.writeValueAsString(obj) for serialization and readValue(json, Class) for deserialization. Gson: new Gson().toJson() and fromJson().
>
> **Example:** Convert the GET /products/101 response JSON into a Product object and assert its price.

```
ObjectMapper om = new ObjectMapper();
Product p = om.readValue(json, Product.class);
```

58. Implement a BaseTest class for API tests.

Answer: A BaseTest sets common API setup once: base URI, authentication, request specification and cleanup, which all test classes inherit.

Example: OrderApiTest extends BaseTest and reuses the base URL and auth token.

```
public class BaseTest {
  @BeforeClass public void setup() {
    RestAssured.baseURI = Config.get("api.url");
  }
}
```

59. Design a config.properties management class.

Answer: Load config.properties once using the Properties class in a static block and expose a static getter, so every class reads values from one place.

Example: Keep url, browser and timeout in config.properties; ConfigReader.get("browser") returns the value.

```
public class ConfigReader {
  private static Properties p = new Properties();
  static {
    try (FileInputStream f = new FileInputStream("config.properties")) {
      p.load(f);
    } catch (IOException e) { throw new RuntimeException(e); }
  }
  public static String get(String k) { return p.getProperty(k); }
}
```

60. What is a Singleton class in Java and where would you use it in a framework?

Answer: Singleton allows only one instance per JVM: private constructor plus a static getInstance(). Use it for config readers, DB connections or report managers. For WebDriver in parallel runs, use ThreadLocal instead.

Example: A single ConfigReader instance shared by all tests, so the properties file is loaded once.

```
class ConfigReader {
  private static ConfigReader inst;
  private ConfigReader() { }
  static synchronized ConfigReader getInstance() {
    if (inst == null) inst = new ConfigReader();
    return inst;
  }
}
```

61. Which design patterns do you know and how do they relate to OOP (e.g. Page Object, Singleton, Factory)? **[asked x2 in your bank]**

Answer: Page Object (page classes with locators/actions), Singleton (config), Factory (creates a driver by browser name), Builder (test data). They rely on encapsulation, abstraction and polymorphism.

Example: Factory returns Chrome or Firefox driver; Page Object classes for Login, Cart and Checkout; Builder creates Order test data.

62. How is multithreading used in automation (parallel execution)?

Answer: Run tests in parallel with TestNG (parallel and thread-count) and keep each thread's browser in ThreadLocal<WebDriver> so sessions do not clash. Scale with Selenium Grid.

Example: Run the cart, search and payment classes in parallel to cut the regression suite time.

```
private static ThreadLocal<WebDriver> tl = new ThreadLocal<>();
tl.set(new ChromeDriver());   WebDriver d = tl.get();
```

63. What is the difference between Apache POI and JXL?

Answer: Apache POI reads/writes both .xls and .xlsx and is actively maintained. JXL supports only older .xls files and is no longer maintained.

Example: Read checkout test data from an .xlsx sheet using POI; JXL cannot open that file.

Core Language: static, final, Access Modifiers & Basics (3)

64. What is the final keyword and what are its uses? (also: static vs final) **[asked x4 in your bank]**

> **Answer:** final variable = constant, final method = cannot be overridden, final class = cannot be extended. static means one copy per class; final means the value cannot change; static final is a constant.
>
> **Example:** static final String BASE_URL = "https://shop.com" shared and unchangeable across tests.

```
public static final int MAX_WAIT = 20;
```

65. What are access modifiers in Java and their scope? **[asked x6 in your bank]**

> **Answer:** private: same class only. default (no modifier): same package. protected: same package plus subclasses. public: everywhere.
>
> **Example:** Locators private in a page class, action methods public, helper methods protected in BasePage for child pages.

66. Is Java pass-by-value or pass-by-reference? **[asked x3 in your bank]**

> **Answer:** Java is always pass-by-value. For objects the reference is copied, so changes to the object's state are visible to the caller, but reassigning the reference is not.
>
> **Example:** addItem(cart) modifies the caller's cart; cart = new Cart() inside the method does not change the caller's cart.

```
void change(Cart c) {
  c.add("A");        // caller sees this change
  c = new Cart();    // caller does not see this
}
```

Tricky "Can We...?" Questions (6)

67. Can we create an object of an abstract class or instantiate an interface? Why not? **[asked x4 in your bank]**

> **Answer:** No. Abstract classes and interfaces may have unimplemented methods. Create objects of a concrete subclass or implementation, or use an anonymous class.
>
> **Example:** new PaymentMethod() fails; new CardPayment() works, or implement the interface inline as an anonymous class.

```
PaymentMethod p = new CardPayment();   // valid
// new PaymentMethod();                  // compile error
```

68. Can we override a static or private method? **[asked x4 in your bank]**

> **Answer:** Static methods cannot be overridden, only hidden (the reference type decides which runs). Private methods are not visible to subclasses, so a same-named method in the child is a new method.
>
> **Example:** A static helper isPriceValid() in the parent and child: calling it through a parent reference runs the parent's version.

69. Can the main method be overloaded or overridden? What if it is private, and can a program run without it? Why is main static? **[asked x7 in your bank]**

> **Answer:** main can be overloaded but not overridden (it is static). The JVM runs only public static void main(String[] args); if private it compiles but the JVM cannot launch it. main is static so the JVM can call it without creating an object. (Very new Java versions relax this, but the classic rule is what is asked.)
>
> **Example:** TestRunner has main(String[] args) to start the suite and an overloaded main(int n) that the JVM ignores.

70. Can we write non-abstract methods in an interface?

> **Answer:** Yes, from Java 8 an interface can have default and static methods with bodies (Java 9 adds private methods).
>
> **Example:** Interface PaymentMethod has default void printReceipt() { } so every implementation gets it for free.

```
interface PaymentMethod {
  void pay();
  default void printReceipt() { System.out.println("Paid"); }
}
```

71. How do you prevent a class from being extended or a method from being overridden? (extending a final class) **[asked x2 in your bank]**

> **Answer:** Declare a class final to prevent extending it (like String) and a method final to prevent overriding. A private constructor also blocks extension. Extending a final class gives a compile-time error.
>
> **Example:** final class PriceCalculator so no test class can change the discount logic.

72. Can we call a non-static variable from a static method?

> **Answer:** Not directly, because a static method has no object. Create an object and access the variable through it.
>
> **Example:** A static main method must create a CartPage object to use its non-static cartCount variable.

```
public static void main(String[] a) {
  Cart c = new Cart();
  System.out.println(c.count);
}
```

Java 8: Lambda, Functional Interfaces & Streams (3)

73. What is a functional interface? **[asked x2 in your bank]**

> **Answer:** A functional interface has exactly one abstract method (default/static allowed) and can be the target of a lambda. Mark it @FunctionalInterface. Examples: Runnable, Comparator, Predicate.
>
> **Example:** Predicate<Product> inStock = p -> p.getStock() > 0 filters available items.

74. What are lambda expressions in Java 8? **[asked x4 in your bank]**

> **Answer:** A lambda is a short anonymous function: (parameters) -> expression. It removes boilerplate for functional interfaces and is common with streams.
>
> **Example:** Print all product names in the search results using one line.

```
products.forEach(p -> System.out.println(p.getName()));
```

75. What is the Stream API? Explain filter, map, flatMap, reduce and collect. **[asked x4 in your bank]**

> **Answer:** Stream API processes collections in a pipeline. filter selects items, map transforms them, flatMap flattens nested collections, reduce combines to one value, collect gathers results.
>
> **Example:** From the cart items, filter in-stock ones, map to price and reduce to the order total.

```
double total = cart.stream().filter(Item::inStock)
  .map(Item::getPrice).reduce(0.0, Double::sum);
```

Multithreading, Synchronization & Garbage Collection (3)

76. What is multithreading and what is a Thread in Java? **[asked x5 in your bank]**

> **Answer:** Multithreading runs multiple threads (independent paths of execution) concurrently in one process. A Thread is created by extending Thread, implementing Runnable or using ExecutorService.
>
> **Example:** Run the search and payment test classes simultaneously, each in its own thread, to shorten execution time.

```
new Thread(() -> runCheckoutTest()).start();
```

77. How do you handle synchronization in multithreading? **[asked x2 in your bank]**

> **Answer:** Use the synchronized keyword (method or block), locks such as ReentrantLock, volatile, atomic classes and concurrent collections; ThreadLocal avoids sharing altogether.
>
> **Example:** A synchronized counter so parallel tests generating order IDs never produce duplicates.
>
> ```
> synchronized int nextId() { return ++id; }
> ```

78. How does Garbage Collection work? Explain memory management and memory leaks. **[asked x3 in your bank]**

> **Answer:** GC automatically frees objects that are no longer referenced; the heap has young and old generations. A memory leak happens when unused objects stay referenced (static collections, unclosed resources). System.gc() is only a request.
>
> **Example:** A browser driver that is never quit keeps objects in memory across tests; driver.quit() releases them.

Section 3: Medium Priority (26 questions)

Breadth topics. Asked occasionally, usually as follow-ups to your earlier answers or to probe a specific area on your resume. One clear paragraph per question is enough.

OOP Concepts (4)

79. What is a class and how is it different from an object?

> **Answer:** A class is a blueprint defining fields and methods; an object is a runtime instance of it with its own state.
>
> **Example:** Class Product is the blueprint; laptop and phone are objects with different names and prices.

80. What are instance variables?

> **Answer:** Instance variables are declared in a class outside methods without static; every object gets its own copy and they get default values.
>
> **Example:** Each Product object has its own name, price and stock instance variables.

81. What is the instanceof operator and how is it used? **[asked x2 in your bank]**

> **Answer:** instanceof checks whether an object is an instance of a class or interface and returns a boolean; commonly used before casting.
>
> **Example:** Check if a payment object is a CardPayment before calling card-specific methods.
>
> ```
> if (payment instanceof CardPayment) { ((CardPayment) payment).verifyCvv(); }
> ```

82. If a class has a user-defined constructor and a main method, which executes first? (class vs abstract class)

> **Answer:** The static main method is the entry point, so it runs first; a constructor runs only when an object is created. An abstract class cannot be instantiated, so its constructor runs only through a subclass object.
>
> **Example:** main in TestRunner starts first; CheckoutPage's constructor runs only when new CheckoutPage(driver) is called.

Collections Framework (3)

83. Explain Queue, Deque and PriorityQueue with examples. **[asked x2 in your bank]**

> **Answer:** Queue is FIFO (LinkedList, ArrayDeque). Deque is double-ended, so add/remove at both ends (ArrayDeque). PriorityQueue orders elements by priority (natural order or Comparator).
>
> **Example:** Queue of orders processed in arrival order; PriorityQueue to process express-delivery orders first.
>
> ```
> Queue<String> q = new LinkedList<>();
> q.add("ORD1"); q.poll();
> ```

84. What is the difference between HashMap and MultiMap?

> **Answer:** HashMap is in the JDK; MultiMap is not (it is in Guava/Apache Commons) and holds multiple values per key. In plain Java use Map<K, List<V>>.
>
> **Example:** Map<String, List<String>> of category -> product names.

85. What are the classes available under the List interface?

> **Answer:** Classes that implement List include ArrayList, LinkedList, Vector and Stack (also CopyOnWriteArrayList for thread-safe use).
>
> **Example:** ArrayList for product results; LinkedList for a browsing history.

Java in Automation Frameworks (4)

86. What are the SOLID principles?

> **Answer:** S: single responsibility, O: open for extension but closed for modification, L: Liskov substitution (subclasses replace parents safely), I: small focused interfaces, D: depend on abstractions, not concrete classes.
>
> **Example:** Each page class has one responsibility (S); tests depend on the PaymentMethod interface rather than CardPayment (D).

87. What is File Handling in Java? How do you read and write a file? **[asked x3 in your bank]**

> **Answer:** Use java.nio.file.Files for simple read/write or BufferedReader/FileWriter for streams; use try-with-resources to close files.
>
> **Example:** Read product SKUs from a text file and write failed order IDs to a results file.

```
List<String> skus = Files.readAllLines(Paths.get("skus.txt"));
Files.write(Paths.get("out.txt"), skus);
```

88. How do you handle dates in Java?

> **Answer:** Use java.time (LocalDate, LocalDateTime, DateTimeFormatter); the older Date, Calendar and SimpleDateFormat are legacy.
>
> **Example:** Compute the expected delivery date as today + 3 days and compare it with the date on the order page.

```
LocalDate d = LocalDate.now().plusDays(3);
String s = d.format(DateTimeFormatter.ofPattern("dd-MM-yyyy"));
```

89. How do you generate random data in Java?

> **Answer:** Use Random or ThreadLocalRandom for numbers, UUID or timestamp for unique strings, and Datafaker/Java Faker for realistic names and emails.
>
> **Example:** Generate a unique email for each new user registration test.

```
String email = "user" + UUID.randomUUID().toString().substring(0, 6)
  + "@test.com";
```

Core Language: static, final, Access Modifiers & Basics (4)

90. What are wrapper classes, autoboxing and unboxing? **[asked x6 in your bank]**

> **Answer:** Wrapper classes (Integer, Double, Boolean) are object forms of primitives. Autoboxing converts a primitive to its wrapper automatically; unboxing does the reverse. Collections need wrappers; unboxing null causes NullPointerException.
>
> **Example:** List<Integer> of quantities uses autoboxing when adding 2; reading it back unboxes to int.

```
List<Integer> qty = new ArrayList<>();
qty.add(2);        // autoboxing
int q = qty.get(0); // unboxing
```

91. What is the difference between primitive and reference data types? Is String primitive? How are data types used in automation? **[asked x3 in your bank]**

> **Answer:** Primitives (int, double, boolean...) store the value directly. Reference types (objects, arrays, String) store an address. String is not primitive; it is a class. In automation: int for timeouts, boolean for isDisplayed(), String for locators, double for prices.
>
> **Example:** int quantity, double price, boolean isInStock and String productName in a Product class.

92. How do you access a default or protected class?

> **Answer:** A default (package-private) class is accessible only within its own package. Protected members are available in the same package and to subclasses in other packages through inheritance. A top-level class cannot be private or protected.
>
> **Example:** A helper class in the cart package without public is used only by classes in that package; a protected method in BasePage is used by page subclasses.

93. What are the main features of Java?

> **Answer:** Object-oriented, platform independent (bytecode on JVM), simple, secure, robust (GC and exception handling), multithreaded, high performance (JIT) and a rich standard library.
>
> **Example:** The same compiled test framework runs on Windows, Linux and the CI server.

Tricky "Can We...?" Questions (4)

94. Can we overload static or private methods? **[asked x2 in your bank]**

> **Answer:** Yes. Overloading is resolved at compile time within a class, so static and private methods can both be overloaded.
>
> **Example:** static String getPrice(String sku) and static String getPrice(String sku, String currency) in a utility class.

95. Is it possible to initialize a variable declared in an interface?

> **Answer:** Interface variables are implicitly public static final, so they must be initialized at declaration and cannot be changed afterwards.
>
> **Example:** interface Limits { int MAX_QTY = 10; } defines the cart limit once.
>
> ```
> interface Limits { int MAX_QTY = 10; }
> ```

96. Can we create a constructor in an abstract class?

> **Answer:** Yes. An abstract class can have constructors; it cannot be used to create an object directly but runs when a subclass object is created (via super()).
>
> **Example:** abstract BasePage(WebDriver driver) constructor initializes the driver for every page that extends it.

97. Can we override a constructor?

> **Answer:** No. Constructors are not inherited, so they cannot be overridden. They can only be overloaded.
>
> **Example:** CartPage cannot override BasePage's constructor but can define its own and call super(driver).

Java 8: Lambda, Functional Interfaces & Streams (2)

98. Explain the Collectors class and its common methods. **[asked x2 in your bank]**

> **Answer:** Collectors provides ready-made collectors for stream.collect(): toList, toSet, toMap, joining, groupingBy, partitioningBy, counting, summingInt and averagingDouble.
>
> **Example:** Group products by category to verify the filter counts.
>
> ```
> Map<String, List<Product>> byCat =
>   products.stream().collect(Collectors.groupingBy(Product::getCategory));
> ```

99. What is the difference between a lambda expression and an anonymous class?

> **Answer:** A lambda only implements a functional interface, is concise, and this refers to the enclosing class. An anonymous class can implement any interface/abstract class with multiple methods and has its own this and state.
>
> **Example:** A one-line lambda for a Comparator versus a multi-line anonymous class for an interface with several methods.

Multithreading, Synchronization & Garbage Collection (2)

100. What is the difference between a synchronized block and a synchronized method?

> **Answer:** A synchronized method locks the whole method on the object (or class if static). A synchronized block locks only the critical section and can use a chosen lock object, so it is faster and more flexible.
>
> **Example:** Lock only the line that updates the shared order counter instead of the whole checkout method.
>
> ```
> synchronized (lock) { orderCount++; }
> ```

101. What is the purpose of the volatile keyword?

> **Answer:** volatile guarantees that a variable's latest value is visible to all threads and prevents caching and reordering, but it does not make compound operations atomic.
>
> **Example:** A volatile boolean stop flag that tells a monitoring thread to end when the test run completes.
>
> ```
> private volatile boolean running = true;
> ```

Arrays & Coding Problems (3)

102. How do you reverse a singly linked list?

> **Answer:** Use three pointers (prev, current, next) and reverse each node's link while walking the list in O(n) time and O(1) space.
>
> **Example:** Reverse a list of recently viewed products to show the newest first.
>
> ```
> Node prev = null, cur = head;
> while (cur != null) { Node n = cur.next; cur.next = prev; prev = cur; cur = n; }
> head = prev;
> ```

103. How do you check whether an array is empty or null?

> **Answer:** Check null first, then length: arr == null || arr.length == 0. For a list use list == null || list.isEmpty().
>
> **Example:** A search for a rare keyword returns an empty result array; the check prevents an exception before accessing the first item.
>
> ```
> if (arr == null || arr.length == 0) { /* empty */ }
> ```

104. How do you define a dynamic array?

> **Answer:** Java arrays have a fixed size, so use an ArrayList for a dynamic array; it grows automatically.
>
> **Example:** Collect the products returned by search into an ArrayList because the count is unknown in advance.
>
> ```
> List<String> items = new ArrayList<>();
> items.add("Phone");
> ```

Section 4: Low Priority / Quick Revision (12 questions)

Fresher-level definitions and niche topics. Rarely asked at 4+ years, but worth a one-line answer so you are never caught off guard.

Collections Framework (1)

105. What features or methods do all classes in the Collection Framework share?

> **Answer:** All Collection classes share methods from the Collection interface: add(), remove(), contains(), size(), isEmpty(), clear(), addAll(), iterator(), toArray() and stream().
>
> **Example:** cart.size() and cart.isEmpty() work the same on any Collection holding cart items.

Core Language: static, final, Access Modifiers & Basics (4)

106. What are Java operators?

> **Answer:** Operators perform operations: arithmetic (+ - * / %), relational (== != < >), logical (&& || !), assignment (=, +=), unary (++ --), ternary (?:) and bitwise.
>
> **Example:** total = price * qty; boolean eligible = total > 500 && isMember; discount = eligible ? 10 : 0.

107. Explain conditional statements and the difference between if-else and switch-case. **[asked x2 in your bank]**

> **Answer:** if-else evaluates any boolean condition or range. switch-case compares one value (int, char, String, enum) against fixed cases and is cleaner for many discrete options.
>
> **Example:** switch on the browser name in a driver factory; if-else for price ranges.

```
switch (browser) {
  case "chrome": d = new ChromeDriver(); break;
  case "firefox": d = new FirefoxDriver(); break;
}
```

108. What are loops, how are they used in QA, and what is the difference between break and continue? **[asked x2 in your bank]**

> **Answer:** Loops repeat code (for, while, do-while, for-each). In QA they iterate over elements and data rows. break exits the loop completely; continue skips the current iteration.
>
> **Example:** Loop through product cards, break once the required product is found, continue on out-of-stock cards.

```
for (WebElement c : cards) {
  if (c.getText().contains("Laptop")) { c.click(); break; }
}
```

109. What is System.out.println and what is it used for?

> **Answer:** It prints text to the console and moves to a new line. System is a class, out is a PrintStream and println is its method. Prefer a logger (Log4j) in frameworks.
>
> **Example:** System.out.println("Order placed: " + orderId) shows the order ID in the console while debugging.

Tricky "Can We...?" Questions (1)

110. Can multiple catch blocks exist without a try block? Will it give a compile-time error?

> **Answer:** No. A catch block must follow a try; without it the code gives a compile-time error. A try needs at least one catch or a finally.
>
> **Example:** catch (Exception e) { } written alone in checkout code will not compile.

Java 8: Lambda, Functional Interfaces & Streams (1)

111. What is the difference between Collection.stream() and Collection.parallelStream()? **[asked x2 in your bank]**

> **Answer:** stream() processes elements sequentially in one thread. parallelStream() splits the work across threads using the common ForkJoin pool; ordering is not guaranteed, so avoid it with shared mutable state or small data.
>
> **Example:** parallelStream for validating a large list of 10,000 product prices; stream for a small cart.

```
prices.parallelStream().allMatch(p -> p > 0);
```

Arrays & Coding Problems (5)

112. What is an array in Java?

> **Answer:** An array is a fixed-size, indexed collection of same-type elements; the index starts from 0 and length gives the size.
>
> **Example:** int[] stock = {5, 0, 12}; holds stock counts for 3 products.

```
int[] stock = {5, 0, 12};
System.out.println(stock.length);   // 3
```

113. What is an anonymous array? Give an example.

> **Answer:** An anonymous array has no name; it is created and passed inline with new Type[]{...}.
>
> **Example:** calculateTotal(new double[]{499, 299, 150}) passes cart prices without declaring a variable.

```
int sum = total(new int[]{499, 299, 150});
```

114. What is the difference between int[] a and int a[]?

> **Answer:** No difference; both declare an int array and int[] a is the preferred style. They differ only in multiple declarations: int[] a, b makes both arrays; int a[], b makes only a an array.
>
> **Example:** int[] prices, discounts; creates two arrays.

115. What are jagged arrays? Give an example.

> **Answer:** A jagged array is a multidimensional array whose rows have different lengths.
>
> **Example:** Each order row holds a different number of items: order 1 has 2 items, order 2 has 4.

```
int[][] orders = new int[2][];
orders[0] = new int[2];
orders[1] = new int[4];
```

116. When creating multidimensional arrays, can you specify a dimension after an empty dimension?

> **Answer:** No. Only the first dimension is mandatory; once a dimension is left empty, later dimensions cannot be specified. new int[3][] is valid; new int[][3] is a compile error.
>
> **Example:** int[][] grid = new int[3][]; is valid for row-wise item counts in orders.